Recommended Practices for Applying Software Updates to a RHEL High Availability or Resilient Storage Cluster

Updated January 30 2024 at 9:31 PM - English

Introduction

With one of the primary responsibilities of a High Availability or Resilient Storage cluster being to provide continuous service for applications or resources, it is especially important that updates be applied in a systematic and consistent fashion to avoid any potential disruption to the availability of those critical services. This document aims to outline Red Hat's recommended practices for applying updates to the cluster software itself and to the software comprising the base RHEL operating system, libraries, and utilities.

Environment

- Red Hat Enterprise Linux (RHEL) 5, 6, 7, 8 or 9 with the High Availability or Resilient Storage Add On
- One or more pieces of software installed on cluster nodes or remote nodes must be updated

Contents

- Updating Software Packages in a RHEL High Availability and/or Resilient Storage Cluster

- Important Notes
- General Overview of Update Procedures
- Risks and Considerations
- Procedure for Rolling Updates
  - RHEL 6, 7, 8 and 9 Clusters using pacemaker
  - RHEL 5 and 6 with cman (no pacemaker)
- Procedure for Entire Cluster Update

Updating Software Packages in a RHEL High Availability and/or Resilient Storage Cluster

Important Notes

- **WARNING**: It is critical when performing software-update procedures for RHEL High Availability and Resilient Storage clusters to ensure that any node that will undergo updates is not an active member of the cluster before those updates are initiated. Swapping out the software that the cluster stack relies on while it is in use can lead to various problems and unexpected behaviors, including but not limited to issues that can cause complete outages of the cluster and services it is managing.

- Red Hat does not support in-place upgrades or rolling-upgrades of cluster nodes, remote nodes, and bundle container images from one major release of RHEL to another *except for RHEL 8.8+ to RHEL 9.2+ as noted below*. For example, there is no supported method for updating some nodes in a cluster from RHEL 6 to RHEL 7, introducing them into the cluster with existing RHEL 6 nodes to take over resources from them, and then updating the remaining RHEL 6 nodes. Upgrades in major releases of RHEL must be done by migrating services from a running cluster on the old release to another cluster running the new release.

  - Upgrade of systems using High Availability add-on from RHEL 6 to RHEL 7 is unsupported.
  - Upgrade of systems using High Availability add-on from RHEL 7 to RHEL 8 is unsupported .
  - Upgrade of systems using High Availability add-on from RHEL 8 to RHEL 9 is generally unsupported except for the following:

    - The only supported rolling upgrades of cluster nodes, remote nodes, and bundle container images is: RHEL 8.8+ to RHEL 9.2+.
    - The limitations and procedure for upgrading clusters nodes from RHEL 8.8+ to RHEL 9.2+ are outlined in the following article: Procedure to upgrade a RHEL 8 High Availability cluster to RHEL 9
    - Do note that it is **not supported** to do rolling upgrade on cluster nodes that have Resilient Storage packages installed.
- Red Hat does **not support** rolling upgrades of shared storage that is exported with `samba+ctdb` : Does ctdb shared storage support rolling upgrades?

- While in the process of performing an update, do not make any changes to your cluster configuration. For example, do not add or remove resources or constraints.

- Although it is not required, when upgrading a Pacemaker cluster it is a good practice to upgrade all cluster nodes **before** upgrading any Pacemaker Remote nodes or podman (Docker) containers used in bundles.

- Please feel free to contact Red Hat Global Support Services for assistance in planning an update, upgrade, or migration of any kind. Proper planning and risk mitigation is key to a successful update or migration, and Red Hat's experts can assist in ensuring the process goes as smoothly as possible.

General Overview of Update Procedures

Updating packages that make up the RHEL High Availability and Resilient Storage Add-Ons, either individually or as a whole, can be done in one of two general ways:

1) *Rolling Updates*: The basic idea is to take a fully-formed and active cluster, remove one node from service by stopping its relevant services and daemons, update its software, then integrate it back into the cluster before repeating the procedure on another node. This allows for the cluster to continue providing service and managing resources while each node is updated, and allowing the update node(s) to provide service while bringing the remaining node(s) up to the same software level. The node undergoing an update at each stage should not be a member of the cluster while the update is ongoing.

2) *Entire Cluster Update*: When a cluster is able to undergo a complete outage, it can simplify update procedures greatly. Such situations allow for stopping the entire cluster, applying updates to all nodes simultaneously (or one-after-another, if preferred), and then starting the cluster back up together. One of the primary benefits of such a procedure is that there is no time when nodes should be running separate versions of the software, thereby eliminating any risk of incompatibilities or unexpected behavior due to such mismatches. This option also eliminates any complexity that might exist with repeatedly moving resources around in the cluster to accommodate each node stopping and then rejoining.

Risks and Considerations

- When performing a *Rolling Update*, the presence of different versions of the High Availability and Resilient Storage within the same cluster introduces a risk that there may be unexpected behavior. While Red Hat does seek to eliminate any known incompatibilities between different releases within the same major release of RHEL, it also only performs limited testing of different versions of the software operating simultaneously. It is always possible that some previously unforeseen incompatibility between versions could cause unexpected behavior, so the only way to completely eliminate this risk is to use the *Entire Cluster Update* method.

- New software versions always come with the potential for unexpected behavior, changes in functionality that may require advance preparation, or in rare cases, bugs causing that could impact the operation of the product. Red Hat strongly recommends having a test, development, or staging cluster configured identically to any production clusters, and using such a cluster to roll out any updates to first for thorough testing prior to the roll-out in production.

- Performing a *Rolling Update* necessarily means reducing the overall capacity and redundancy within the cluster. The size of the cluster dictates whether the absence of a single node poses a significant risk, with larger clusters obviously being able to absorb more node failures before reaching the critical limit, and with smaller clusters being less capable or not capable at all of withstanding the failure of another node while one is missing. It is important that the potential for failure of additional nodes during the update procedure be considered and accounted for. If at all possible, taking a complete outage and updating the cluster entirely may be the preferred option so as to not leave the cluster operating in a state where additional failures could lead to an unexpected outage.

Procedure for Rolling Updates

The specific steps to follow differs depending on the RHEL release and style of cluster in use.

- Only run `pcs` commands on updated nodes whenever possible. The newer version should always be backward-compatible, but the older version may not be forward-compatible.
- Once the last active older cluster node has been taken out of the cluster, no older cluster node will be able to rejoin without being updated first.

RHEL 6, 7, 8 and 9 Clusters using pacemaker

Perform the following steps to update the base RHEL packages, High Availability Add-On packages, and/or Resilient Storage Add-On packages on each node in a rolling-fashion:

1) Choose a single node where the software will be updated, to help reduce downtime we would recommend first migrating nodes running the smallest amount of services or a cluster node that is the passive cluster node for *promotable* pacemaker managed resources. If any preparations need to be made before stopping or moving the resources or software running on that node, carry out those steps now .

2) Before starting the procedure it is recommended that the systemd service `pacemaker` is disabled from starting at boot. Disabling `pacemaker` from starting at boot will prevent `pacemaker` from starting **until** verifying that all components managed by the cluster actually still work as expected.

The cluster stack can be disabled from starting on boot on this chosen node with:

```
# # Syntax: # pcs cluster disable [<node>]
# # Example:
# pcs cluster disable node1.example.com
```

Enabling `pacemaker` to start at boot should only be enabled after it has been verified that the cluster is still able to manage all the clustered managed resources without issues.

3) If the cluster is composed of 3 or more cluster nodes then move any managed resources off of this node as needed. If there are specific requirements or preferences for where the resources should be relocated to, then consider creating new location constraints to place the resources on the correct node. The location of resources can be strategically chosen to result in the least number of moves throughout the *Rolling Update* procedure, rather than moving resources in preparation for every single node update.

Otherwise if allowing the cluster to automatically manage placement of resources on its own is acceptable, then the next step will automatically take care of this.

4) Place the chosen node in standby mode to ensure it is not considered in service, and to cause any remaining resources to be relocated elsewhere or stopped.

```
# # Syntax: # pcs node standby [<node>]
# # Example:
# pcs node standby node1.example.com
```

5) Stop the cluster software on the chosen node using `pcs` :

```
# # Syntax: # pcs cluster stop [<node>]
# # Example:
# pcs cluster stop node1.example.com
```

6) Perform any necessary software updates on the chosen node. There are various methods for doing so that are outside the scope of this article. Consult the general instructions for installing High Availability and Resilient Storage software, Knowledge Content in the Customer Portal, and/or the Product Documentation. After a cluster node has been updated it must be manually verified that all components running on the node that are managed by the cluster are still working as expected to avoid unnecessary failures to be triggered when the cluster node is a member of the cluster managing cluster resource when the cluster stack is started again on the cluster node.

7) If any software was updated that necessitates a reboot, prepare to perform that reboot. It is recommended that cluster software be disabled from starting on boot so that the host can be checked to ensure it is fully functional on its new software versions before bringing it into the cluster as noted in *step 2*.

Perform the reboot when ready, and when complete, ensure the host seems to be fully functional and is using the correct software in any relevant areas (such as having booted into the latest kernel). If anything does not seem correct, then do not proceed until the situation is resolved. Contact Red Hat Global Support Services for assistance if needed.

8) Rejoin the updated node into the cluster.

```
# # Syntax: # pcs cluster start [<node>]
# # Example:
# pcs cluster start node1.example.com
```

Check `pcs status` output to determine if everything appears as it should. Once the node seems to be functioning properly, reactivate it for service by taking it out of standby mode:

```
# # Syntax: # pcs node unstandby [<node>]
# # Example:
# pcs node unstandby node1.example.com
```

9) If any temporary location constraints were created in step 2 to control the placement of resources, then adjust or remove them to allow resources to go back to their normally preferred locations.

10) After verifying that all `pacemaker` managed cluster resources are able to run on the cluster node then enable `pacemaker` to start at boot.

```
# # Syntax: # pcs cluster enable [<node>]
# # Example:
# pcs cluster enable node1.example.com
```

11) Repeat steps 1-8 for each remaining node.

12) After all the cluster nodes or remote nodes have been upgraded then run the following:

```
# pcs cluster cib-upgrade
```

RHEL 5 and 6 with cman (no pacemaker)

Perform the following steps to update the base RHEL packages, High Availability Add-On packages, and/or Resilient Storage Add-On packages on each node in a rolling-fashion:

1.) Choose a single node where the software will be updated. If any preparations need to be made before stopping or moving the resources or software running on that node, carry out those steps now.

2.) Move any managed resources off of this node as needed. If there are specific requirements or preferences for where resources should be relocated to, then consider moving them with `clusvcadm`, the Conga web administration interface, or `ccs` (RHEL 6 only) to place the resources on the correct node. Otherwise if allowing the cluster to manage placement of resources on its own is acceptable, then the next step will automatically take care of this.

3.) Stop all running cluster daemons on this chosen node. In RHEL 6 this can be done easily using `ccs`:

```
# # Syntax: # ccs -h <hostname> --stop
# # Example:
# ccs -h node1.example.com --stop
```

NOTE: This also disables the cluster daemons from starting on boot.

4.) Perform any necessary software updates. There are various methods for doing so that are outside the scope of this article. Consult the general instructions for installing High Availability and Resilient Storage software, Knowledge Content in the Customer Portal, and/or the Product Documentation.

5.) If any software was updated that necessitates a reboot, prepare to perform that reboot. It is recommended that cluster software be disabled from starting on boot so that the host can be checked to ensure it is fully functional on its new software versions before bringing it into the cluster. The cluster daemons can be disabled from starting on boot on this chosen node with `chkconfig <service> off`, or using `ccs` as seen in step 3 above.

Perform the reboot when ready, and when complete, ensure the host seems to be fully functional and is using the correct software in any relevant areas (such as having booted into the latest kernel). If anything does not seem correct, then do not proceed until the situation is resolved. Contact Red Hat Global Support Services for assistance if needed.

Once everything appears to be set up correctly, re-enable the cluster daemons on this node using `chkconfig`, or with `ccs` as described in step 6 below.

6.) Rejoin the updated node into the cluster by starting the cluster daemons, or by using `ccs` in RHEL 6:

```
# # Syntax: # ccs -h <hostname> --start
# # Example:
# ccs -h node1.example.com --start
```

NOTE: This enables cluster daemons to start automatically on boot.

Once all daemons are started, the node should be fully functional and managing resources within the cluster.

7.) Repeat steps 1-6 for each remaining node.

Procedure for Entire Cluster Update

The process for updating an entire cluster at once is nearly identical to the *Rolling Update* procedure above, with the single difference being that each step should be performed on all nodes before moving on to the next step. So, for example, stop the cluster daemons on each node before moving on to updating the software, and reboot each node before moving on to re-enabling the cluster software, etc. In the end, the goal is to stop the cluster software on all nodes, update those nodes, then start the cluster software again. The above steps can be used as a guide, and may even be simplified to skip some of the preparation steps if they are not required.

Product(s) **Red Hat Enterprise Linux** **Category** **Install** **Component** **cluster**

Tags **cluster ha high availability** **rhel_5** **rhel_6** **rhel_7** **rhel_8** **rhel_9** **upgrade** **Article Type** **General**

Was this helpful?

YES NO

People who viewed this article also viewed

What are the recommended best practices for mkfs.gfs2?

Solution - Jun 15, 2024

Recommended practices for JBoss clustering

Solution - Aug 7, 2024

Recommended Practices for OpenStack

Article - Jun 19, 2017

Get notified when this content is updated FOLLOW

Comments



Add comment

Formatting Help

Send notifications to content followers

Submit



NEWBIE

16 Points

Prashant

Aug 30, 2018 2:50 AM

Was helpful, understanding the flow.

Reply



NEWBIE

5 Points

GORKA

Oct 31, 2019 9:49 PM

Hi

Which is the difference on RHEL 6 between using "ccs -h --startall" and issuing "service cluster start" ?

Reply



RED HAT

GURU

1356 Points

Reid Wahl

Oct 31, 2019 10:32 PM

Does your RHEL 6 system have a service called `cluster`? Mine does not.

Reply

COMMUNITY MEMBER

57 Points

Felipe Pereira

Apr 1, 2021 5:28 AM

EDIT: in fact, there is a 'pcs cluster standby' command; it is just not documented (pcs cluster --help | grep -i standby). I thought the only standby command was 'pcs node standby'. I don't know if they are the same, though.

Reply



RED HAT

GURU

1356 Points

Reid Wahl

Apr 1, 2021 5:41 AM

Hi, Felipe. Thank you for reporting this.

Until RHEL 8, there was a `pcs cluster standby` command alongside the `pcs node standby` command. They both did the same thing. I see that in RHEL 8, the `cluster` version has been removed.

I've updated the procedure to use the `node` syntax.

Reply

COMMUNITY MEMBER

60 Points

s******

Sep 3, 2021 1:38 PM

While following "Entire Cluster Update" method is it necessary to put entire cluster in standby mode after stopping cluster for update?

Reply



RED HAT

GURU

1356 Points

Reid Wahl

Sep 3, 2021 1:44 PM

Hi, Siva.

You can't put the nodes into standby mode **after** stopping the cluster. This step would be done **before** stopping the cluster.

You're basically correct though, if I understand the point you're making. It's unlikely to make any difference whether you place all the nodes in standby mode before stopping the cluster on all nodes. Stopping the cluster gracefully stops all resources first, so the end result should be the same.

Nonetheless, it doesn't take a significant amount of extra time to put the nodes in standby mode before stopping the cluster. So I would suggest putting them in standby, in case there are any edge cases that I didn't consider.

Reply



COMMUNITY MEMBER

60 Points

s******

Sep 3, 2021 2:59 PM

That clears some doubts, however I am having only two nodes and all I want is to update pacemaker by stopping entire cluster because we can spare 10 - 15 mins downtime. I was able to run "pcs node standby --all" and "pcs cluster stop --all" successfully but got the error when I tried to put them in standby mode and stopped cluster on individual nodes.

So is it a good practice to put both the nodes in standby mode and stop them simultaneously or do i need to go one node after other.

Reply



RED HAT

GURU

1356 Points

Reid Wahl

Sep 3, 2021 3:14 PM

"...got the error when I tried to put them in standby mode and stopped cluster on individual nodes." What was the error? If you have a Red Hat support subscription, I encourage you to open a case to discuss further and ensure timely response.

"So is it a good practice to put both the nodes in standby mode and stop them simultaneously or do i need to go one node after other." If you're performing an entire cluster update (instead of a rolling update), then you would normally put both the nodes in standby simultaneously and stop them simultaneously. It shouldn't matter whether you do it simultaneously or consecutively, however.

Reply

COMMUNITY MEMBER

60 Points

s******

Sep 3, 2021 3:43 PM

Great, thanks for the advice it was really helpful.

Reply



COMMUNITY MEMBER

31 Points

Paul

Dec 9, 2021 3:42 AM

With a 2-node cluster, is there any risk of fencing the active node from 'pcs cluster stop standby-node'?

Reply



RED HAT GURU

1356 Points

Reid Wahl

Dec 9, 2021 4:06 AM

There's not much risk, but it's not zero risk either. It would require some bad luck (or bad configuration I suppose). For example, let's say that you haven't put the nodes in standby mode yet, and you run `pcs cluster stop standby-node`. Then the resources could stop on `standby-node`, attempt to start on `active-node`, fail to start on `active-node`, and then fail to stop on `active-node`. This could cause `active-node` to be fenced.

If you've already put `standby-node` in standby mode before running `pcs cluster stop standby-node`, then that scenario is ruled out. It's hard for me to imagine another scenario that would lead `active-node` to be fenced. You can never 100% guarantee against an edge case though.

Reply



NEWBIE

7 Points

Varun Sharma

Feb 24, 2024 2:35 AM

I'm slightly confused by this article. It tells me they do not support in place upgrades of clusters, but then they give instructions on how to do it. Does that mean you can do it, but if it goes wrong they won't give you support?

Reply





Quick Links

Help

Site Info

Related Sites

About

Red Hat Subscription Value

About Red Hat

Red Hat Jobs



About Red Hat
Jobs
Events
Locations
Contact Red Hat
Red Hat Blog
Diversity, equity, and inclusion
Cool Stuff Store
Red Hat Summit

Copyright © 2024 Red Hat, Inc.
Privacy statement
Terms of use
All policies and guidelines
Digital accessibility
Cookie preferences